Online Food Ordering System Project

Database Systems Design – CSE244

Friday, 21 March 2025

Submitted to:
Professor. Nivin Atef

Eng. Esraa Karam

Submitted by (Team 10):

- Salma Tamer Fathi Moussa 23P0023
- Nada Medhat Serour 23P0272
- Renad Sameh Ibrahim 23P0292
- Esraa Mustafa Elhosini 23P0386
- Mohamed Tarek 2300535
- Rawan Hany Shaker 23P0393

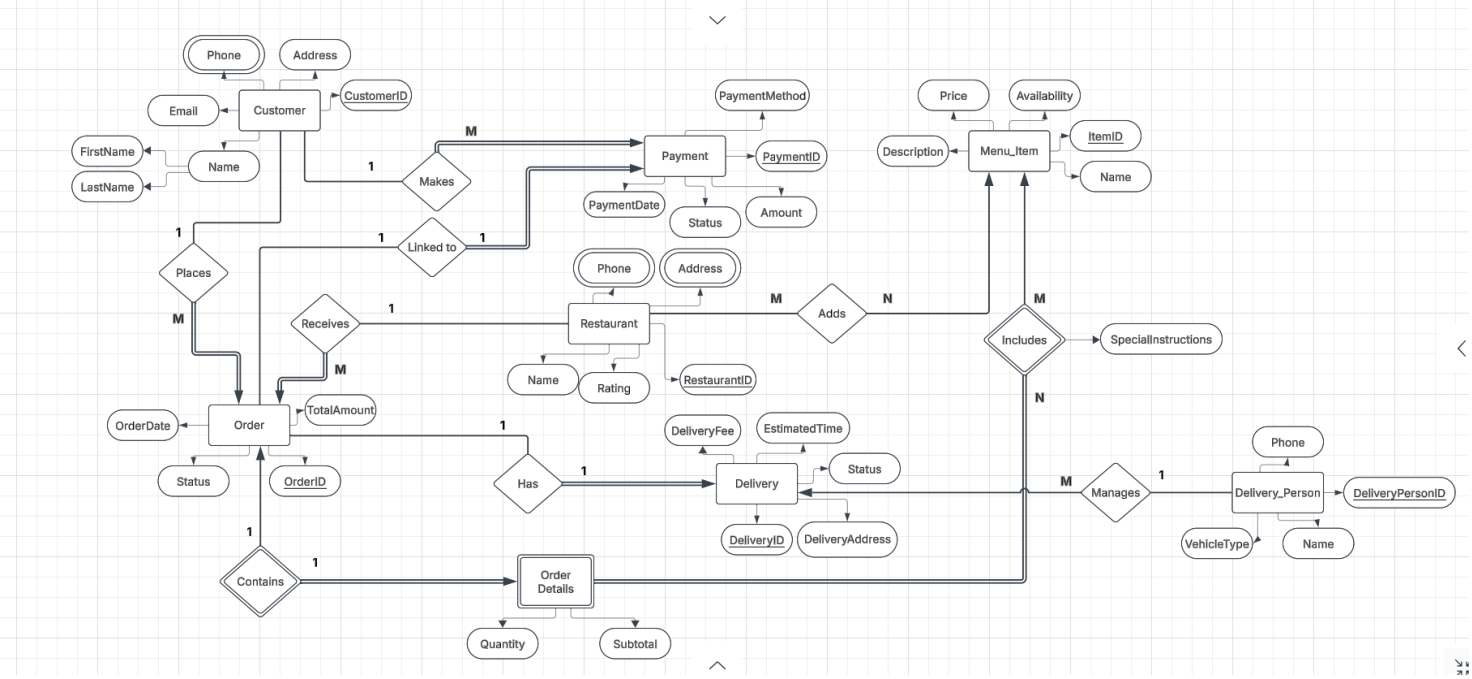
Link to PDF file that contains the ERD and Relational Schema:

ERD 1 (6).pdf

Entity-Relationship Diagram (ERD):

Detailed ERD representing all entities, their attributes, and the relationships between them.

Detailed ERD Graph:



Entities description and their Attributes:

1. Customer: Stores customer personal information (name, email, address).

- **CustomerID**
- **FirstName**
- **LastName**
- **Email**
- **Address**

2. Customer Phone: Allows each customer to have multiple phone numbers.

- **CustomerID**
- **Phone**

3. Order: Captures customer orders, linking them to both restaurants and customers.

- **OrderID**
- **Status**
- **OrderDate**
- **TotalAmount**
- **RestaurantID**
- **CustomerID**

4. Order Details: A linking table detailing which menu items are included in each order along with quantity and subtotal.

- **OrderID**
- **ItemID**
- **Quantity**
- **Subtotal**

5. Restaurant: Contains basic information about each restaurant, including name and rating.

- **RestaurantID**
- **Name**
- **Rating**

6. Restaurant Phone: Stores one or more phone numbers for each restaurant.

- **RestaurantID**
- **Phone**

7. Restaurant Address: Stores one or more addresses for each restaurant (branches).

- **RestaurantID**
- **Address**

8. Menu Item: Represents individual food and drink items offered by restaurants.

- **ItemID**
- **Name**
- **Availability**
- **Price**
- **Description**
- **RestaurantID**

9. Payment: Records payment information, linking customers and orders to payment details (e.g., method, amount).

- **PaymentID**
- **PaymentMethod**
- **Amount**
- **Status**
- **PaymentDate**
- **CustomerID**
- **OrderID**

10. Delivery: Captures delivery details for each order, including status and estimated delivery time.

- **DeliveryID**
- **Status**
- **DeliveryAddress**
- **EstimatedTime**
- **OrderID**

- **DeliveryPersonID**

11. Delivery Person: Stores information about delivery staff such as name, phone, and vehicle type.

- **DeliveryPersonID**
- **Name**
- **Phone**
- **VehicleType**

Relationships:

1. Customer – Order

- **One-to-Many (1:M):**
- One **Customer** can place multiple **Orders**.
- Each **Order** is linked to one **Customer**.
- (**CustomerID** in Order as FK)

2. Customer – Customer Phone

- **One-to-Many (1:M):**
- One **Customer** can have multiple phone numbers.
- Each phone number belongs to one **Customer**.
- (**CustomerID** in Customer Phone as FK)

3. Restaurant – Order

- **One-to-Many (1:M):**
- One **Restaurant** can receive multiple **Orders**.
- Each **Order** is linked to one **Restaurant**.
- (**RestaurantID** in Order as FK)

4. Restaurant – Menu Item

- **Many-to-Many (M:N):**
- Many **Restaurants** can offer multiple **Menu Items**.
- Many **Menu Items** can belong to many **Restaurants**.
- (**RestaurantID** in Menu Item as FK)

5. Restaurant – Restaurant Phone

- **One-to-Many (1:M):**
- One **Restaurant** can have multiple phone numbers.
- Each phone number belongs to one **Restaurant**.
- (**RestaurantID** in Restaurant Phone as FK)

6. Restaurant – Restaurant Address

- **One-to-Many (1:M):**

- One **Restaurant** can have multiple addresses (e.g., multiple branches).
- Each address belongs to one **Restaurant**.
- (**RestaurantID** in Restaurant Address as FK)

7. Order – Order Details

- **One-to-Many (1:M):**
- One **Order** can include multiple **Order Details** (items).
- Each **Order Detail** is linked to one **Order**.
- (**OrderID** in Order Details as FK)

8. Menu Item – Order Details

- **One-to-Many (1:M):**
- One **Menu Item** can appear in multiple **Order Details**.
- Each **Order Detail** links to one **Menu Item**.
- (**ItemID** in Order Details as FK)

9. Customer – Payment

- **One-to-Many (1:M):**
- One **Customer** can make multiple **Payments**.
- Each **Payment** is linked to one **Customer**.
- (**CustomerID** in Payment as FK)

10. Order – Payment

- **One-to-Many (1:M):**
- One **Order** can have one or more **Payments** (in case of split or partial payments).
- Each **Payment** is linked to one **Order**.
- (**OrderID** in Payment as FK)

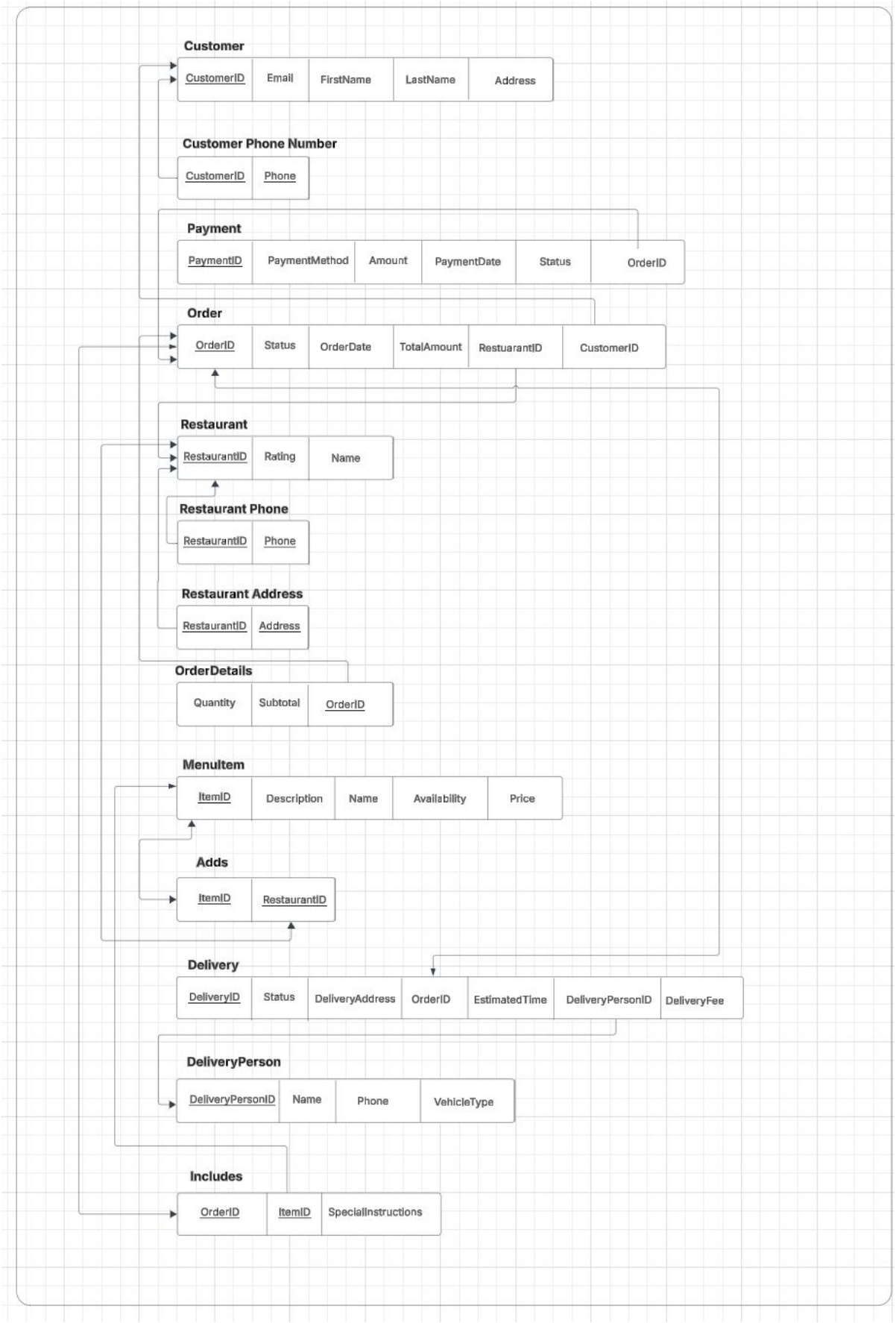
11. Order – Delivery

- **One-to-One (1:1):**
- Each **Order** has exactly one **Delivery**.
- Each **Delivery** is linked to one **Order**.
- (**OrderID** in Delivery as FK)

12. Delivery Person – Delivery

- **One-to-Many (1:M):**
- One **Delivery Person** can handle multiple **Deliveries**.
- Each **Delivery** is assigned to one **Delivery Person**.
- (**DeliveryPersonID** in Delivery as FK)

Relational Schema: *The ERD is converted into a relational schema. The schema is structured logically to maintain efficiency and ensure database integrity.*



1. Customer & Customer Phone Number

- The **Customer** table stores customer details such as ID, name, email, and address.
- A customer may have **multiple phone numbers**, which is why phone numbers are stored separately in the **Customer Phone Number** table. This creates a **1:M (one-to-many)** relationship between Customer and Customer Phone Number.

2. Restaurant, Restaurant Phone & Restaurant Address

- The **Restaurant** table contains the core information about each restaurant like ID, name, and rating.
- A restaurant may have multiple phone numbers and addresses (e.g., multiple branches), so the **Restaurant Phone** and **Restaurant Address** tables store these separately, maintaining a **1:M** relationship for both.

3. MenuItem

- The **MenuItem** table stores all the food or drink items provided by each restaurant.
- Each item has attributes such as name, availability (in stock or not), price, and description.
- Every menu item is linked to a specific restaurant using **RestaurantID** as a foreign key.

4. Order & OrderDetails

- The **Order** table represents an order placed by a customer. It stores information such as order ID, date, total amount, status (e.g., pending, completed), and links to both the **Customer** and **Restaurant**.
- The **OrderDetails** table acts as a **junction table** to resolve the many-to-many relationship between **Orders** and **Menu Items**.
    - For example, one order can have many items, and one menu item can appear in many orders.
    - It includes **OrderID**, **ItemID**, **Quantity**, and **Subtotal** for each item.

5. Payment

- The **Payment** table records the payments made by customers for their orders.
- It includes payment details like method (e.g., cash, card), amount, status (e.g., paid, pending), and payment date.
- Each payment is linked to both a **Customer** and an **Order** through foreign keys.

6. Delivery & DeliveryPerson

- The **Delivery** table tracks delivery-specific information such as delivery status, address, and estimated time.
- Each delivery is linked to one **Order** and one **DeliveryPerson**.
- The **DeliveryPerson** table stores details about delivery staff, such as name, phone, and vehicle type.

Overall Schema Logic (Design Choices):

- The schema follows **normalization rules**, separating repeated data into different tables (e.g., phone numbers and addresses are stored in separate tables).
- The design ensures **data integrity** and **referential consistency** by using **foreign keys** to enforce valid relationships between tables.
- The model supports:
    - Customers placing multiple orders.
    - Orders containing multiple menu items.
    - Restaurants offering many menu items and having multiple contact points (phones & addresses).
    - A flexible delivery and payment process, supporting different delivery persons and multiple payment records.

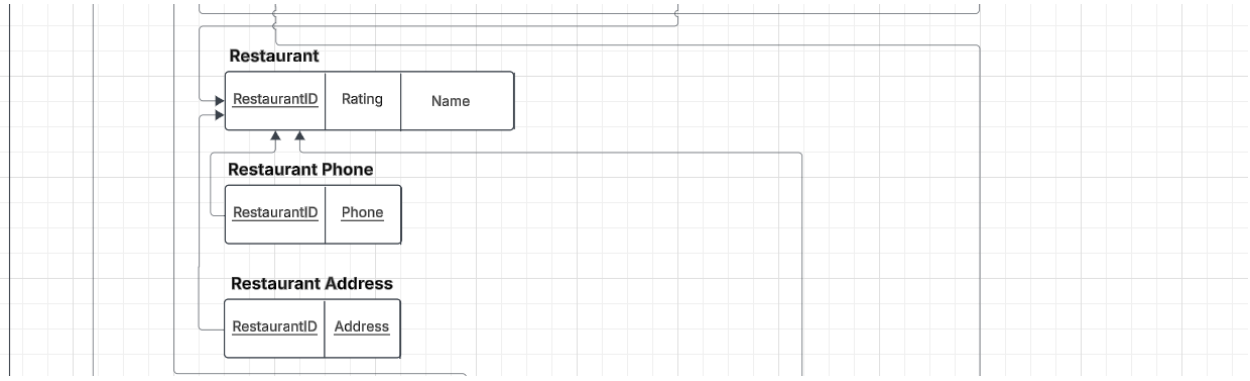
***Normalization**:*

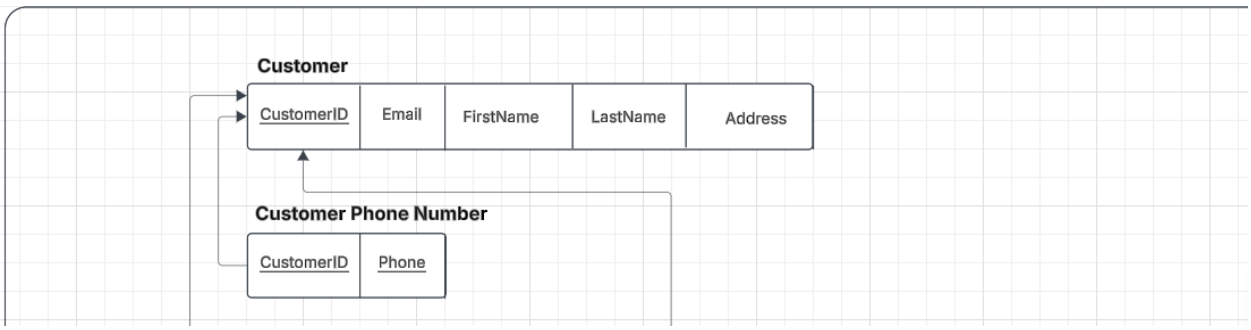
Ensure that the database follows 1NF, 2NF, and 3NF to eliminate redundancy and maintain data integrity. You should provide a brief explanation of how normalization was applied to improve the database structure.

1NF (First Normal Form) – All attributes depend on the key

Requirement: Each column should have atomic (indivisible) values, and each record should be unique.

Application in the Schema:
We placed multivalued attributes such as ***CustomerPhone***, ***RestaurantPhone*** and ***RestaurantAddress*** in separate tables (i.e., ***Customer Phone***, ***Restaurant Phone*** and ***Restaurant Address*** rather than allowing multiple phone numbers and addresses in a single column).
Each attribute contains only a single value per row.

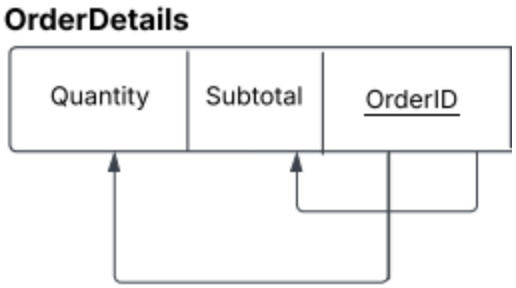




2NF (Second Normal Form) – All attributes depend on the whole key

Requirement: The database must be in 1NF, and all non-key attributes must be fully dependent on the **whole** primary key.

Application in the Schema: The ***OrderDetails*** table, which includes ***OrderID, Quantity,*** and ***Subtotal***, is in 1NF and all the attributes fully depend on ***OrderID***. In the ***Payment*** table, ***CustomerId*** was partially dependent and redundant since it's already in the ***Orders*** table as well, so it was removed from the table in this step.



3NF (Third Normal Form) – All attributes depend on nothing but the key

Requirement: The database must be in 2NF, and all non-key attributes must depend **only** on the primary key.

Application in the Schema: All attributes in all the tables in the schema depend only on the primary key.